Distributed and Mobile Computing Systems
D7024E

Course Introduction

Olov Schelén



Welcome to D7024E!

- Teachers:
  - Johan Kristiansson - Examiner
  - Carl Borngrund
  - Christoffer Fink
  - Olov Schelén (minor role)

Each one of us will introduce ourselves when we meet you for the first time!

Background: A pick of Olov´s current research

- AIMS5.0: Grass mowing as a service – control of automowers and partners in the service chain
  - https://aims50.eu/

- ArtWork : Decision support systems for Assembly Line Balancing in Truck manufacturing
  - https://itea4.org/project/artwork.html

- Stamina & SIM : Secure micro transactions with scalability and authenticity
  - https://www.vinnova.se/en/p/stamina--secure-transactions-and-micro-payments-with-global-network-scaled-authenticity-/

Content

- Course description (walk trough in Canvas)
- Introduction to Distributed Systems
- Primitives for Distributed Communication
- Load balancing, web system use case
- Multiprocessor and Multicomputer Systems
- Event ordering and Consistency of a Distributed System
- Transparencies

Intended learning outcomes

- be familiar with current models and approaches to building mobile and distributed computing systems;
  - objective: what's current and upcoming in software industry!
- have developed practical skills in the use of these models and approaches;
  - objective: hands-on experience by labs and assignments
- have developed understanding of trade offs and ability to select and apply appropriate tools for a particular case;
  - objective: analyze sweet spots and downsides of approaches
- be aware of some current research directions in the field and their possible outcomes.
  - objective: understanding where next steps may be

Teaching and Learning Activities (TLA)

- TLA
  - lectures
  - one large lab assignment
  - student seminars presenting scientific publications
- Three parts are registered, total 7.5 ECTS:
  - lab work 3 ECTS
  - seminar 1.5 ECTS
  - home exam (at scheduled time slot) 3 ECTS

The final course grade is obtained as follows

Weights:
0%: Quizzes (you should take them for learning, but not graded)
55%: Lab assignment
15%: Seminar
30%: Written home exam

The final grade of the course will be determined by mapping the interval 0-100 on a four-grade scale: U (not passed), 3, 4, 5. The **preliminary mapping** is as follows:

- 50 for grade 3,
- 70 for grade 4,
- 85 for grade 5.

Note that the levels are indicative; the final levels depend on the outcome.

Course content

- Architectures of distributed systems: client-server, peer-to-peer, cloud-native
- Communication models, inter-process communication, NAT traversal, and pub-sub
- Serialisation, RPC/REST, and secure transport protocols (TLS, Noise)
- Distributed hash tables (DHTs), gossip protocols, and peer discovery
- Time synchronisation, logical clocks, and global state snapshots
- Data replication and synchronisation: CRDTs, OT, and eventual consistency
- Leader election, consensus algorithms (Paxos, Raft), and coordination services
- Byzantine fault tolerance and blockchains: PoW, PoS, PBFT, HotStuff
- Distributed storage and caching: consistent hashing, erasure coding, CDNs
- Distributed game engines: state sync, lag compensation, prediction techniques
- Virtualisation and compute infrastructure: VMs, containers, WebAssembly, Kubernetes
- Future directions: edge computing, AI-driven orchestration, decentralised agents
- Design trade-offs in distributed systems: consistency, latency, availability, complexity

Lecture plan, see Canvas Syllabus

Please note

- Distributed Systems (DS) are complex
  - The lab will give you in-depth experience on building one kind of DS
  - The lectures will cover essential knowledge on many kinds of DS
- So, listen carefully to the lectures
  - You will get essential knowledge for your future career, in a condensed way
  - Concepts and technologies (use cases) are presented
  - This extends way outside of what is tested in the lab

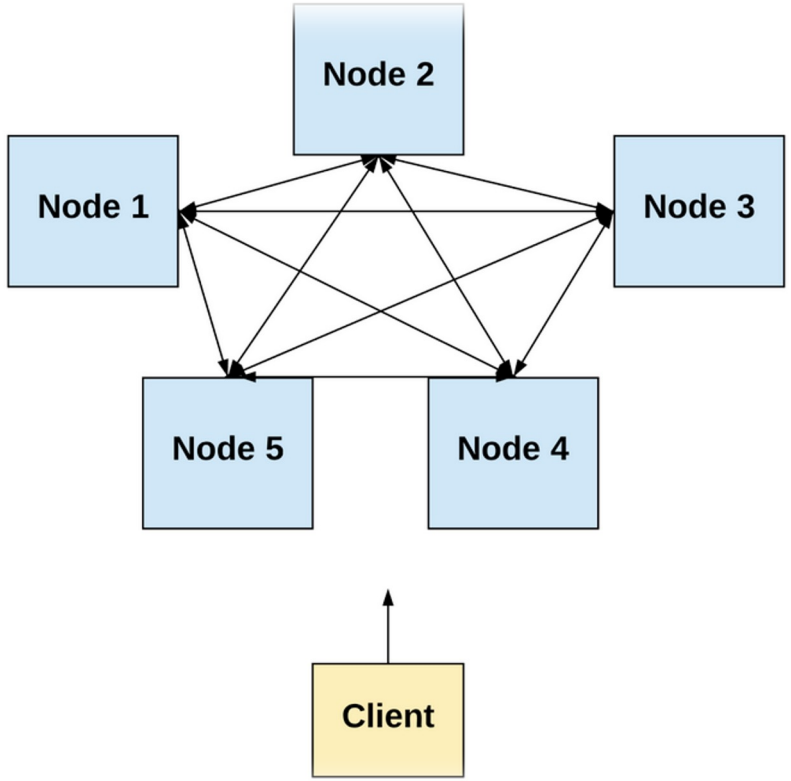
This lecture, outline

- Course description (in Canvas)
- Introduction to Distributed Systems
- Primitives for Distributed Communication
- Load balancing, web system use case
- Multiprocessor and Multicomputer Systems
- Event ordering and Consistency of a Distributed System
- Transparencies

A distributed system

- A service is provided by many cooperating ***nodes*** (roughly computers, virtual machines, or containers)
  - Distributed in the “cloud”
  - Distributed in the “field”
  - Client-cloud or peer-to-peer

- It appears like a single service
  - most internet-based services are provided by distributed systems



A Distributed System……

- ..... can be characterized as a collection of autonomous nodes using a network for communication:
  - No common clock
  - No shared memory
  - Geographical separation (or at least over racks)
    - Communication speed in *not negligible*
    - Internet wide or cluster based (very different conditions)
  - Autonomy and heterogeneity (loosely coupled)
    - Different speed and varying storage capacity
    - May even have different operating systems

Software makes the difference

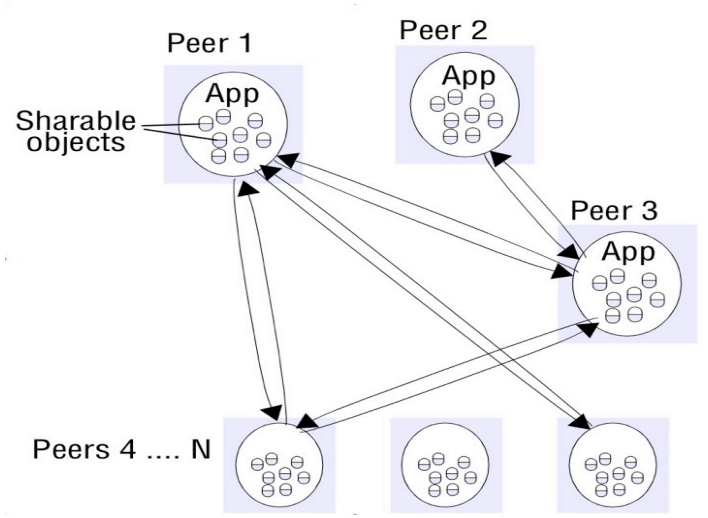
- Software defined everything
- Hardware is streamlined
  - Change software on a piece of hardware and you get a completely different product
- However, “visible” devices must have nice design
  - software and hardware are often bundled



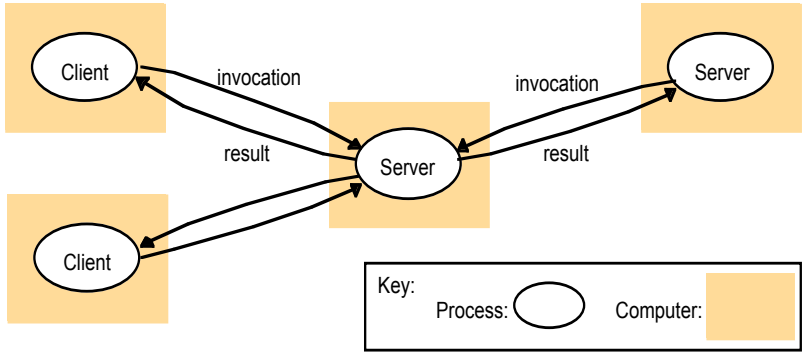


Distributed system models

- Peer-to-Peer (P2P)

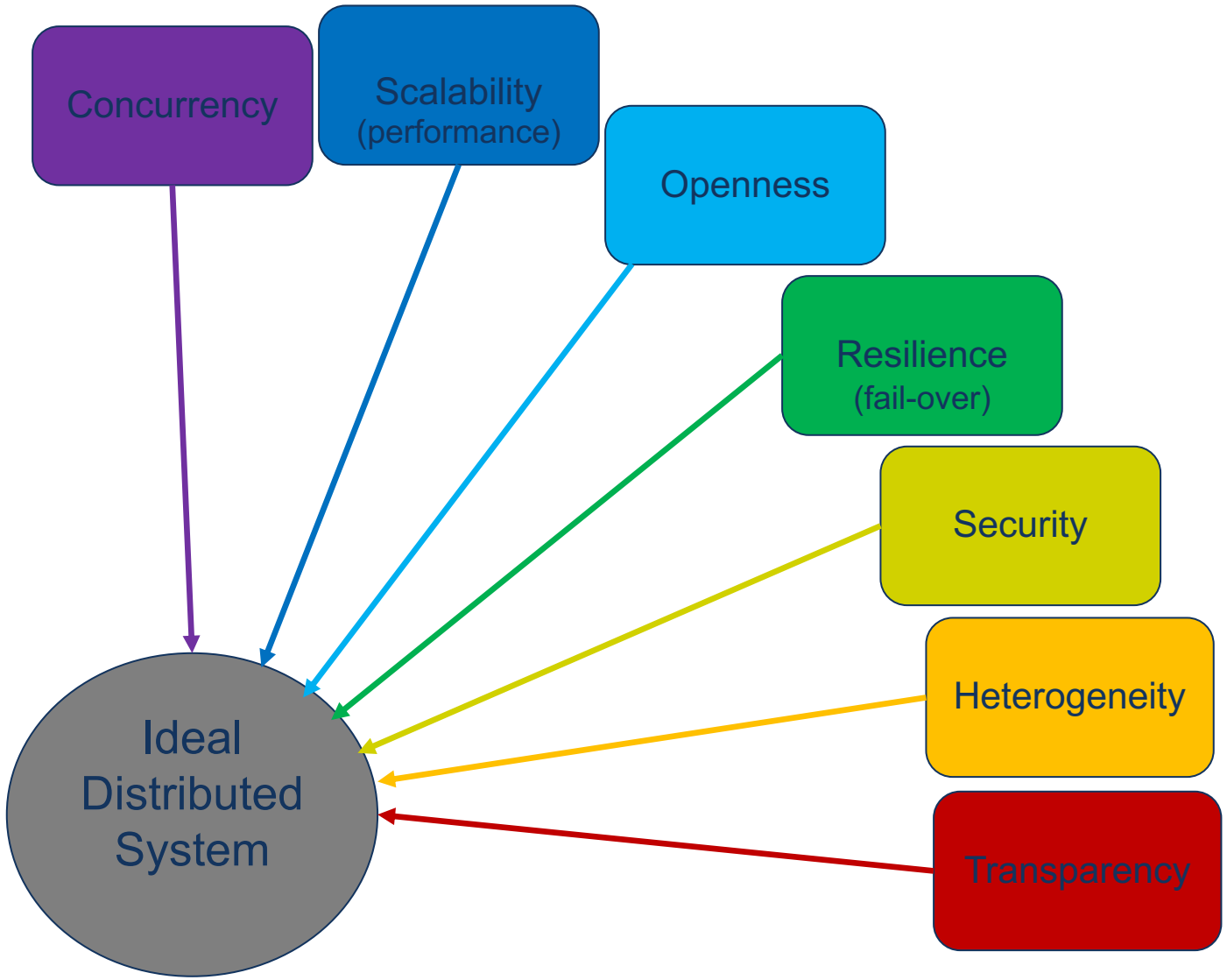


- Client-Server also Client-Cloud



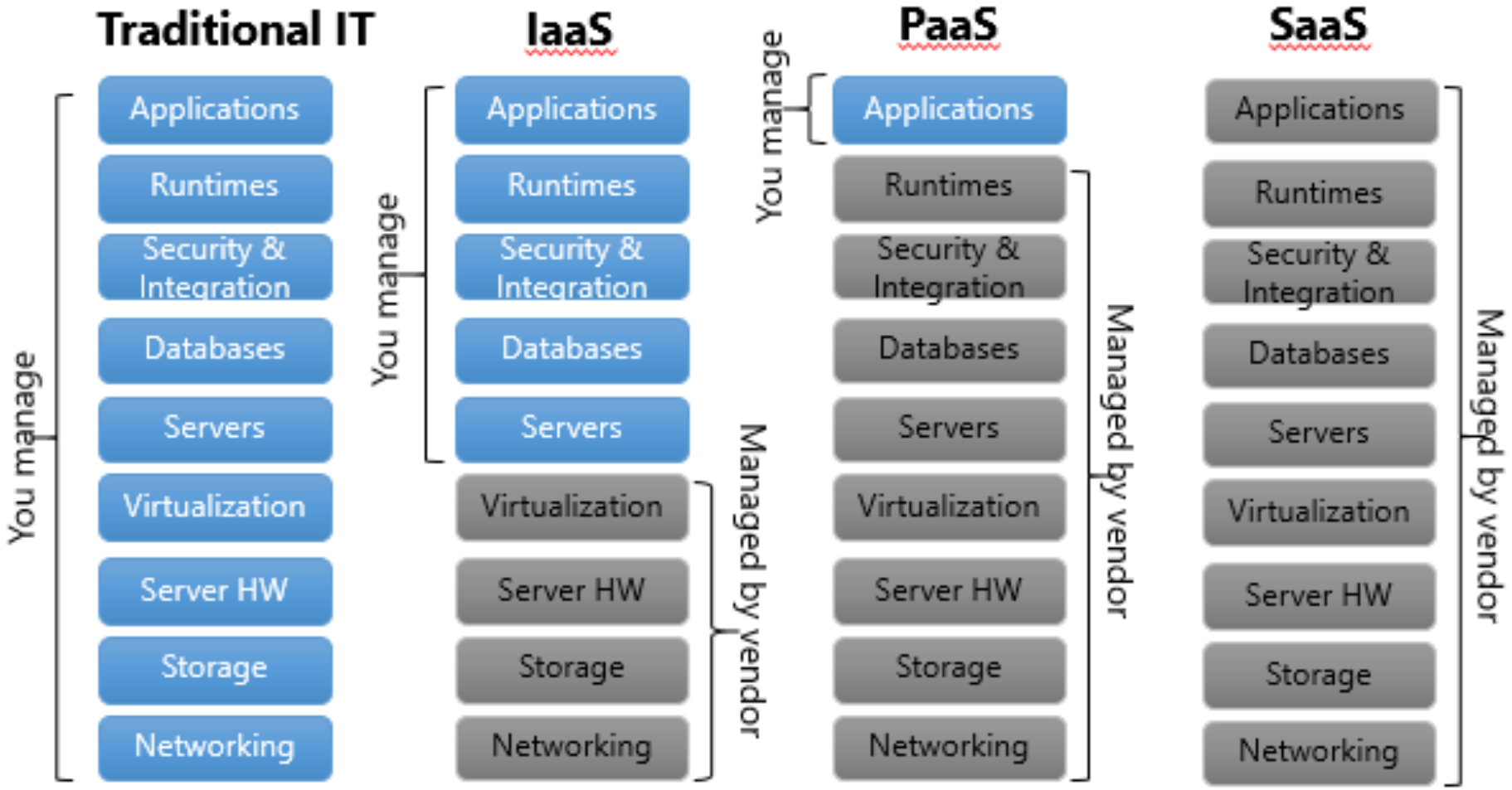
Often combined. The "Server" is really like the above.

Properties

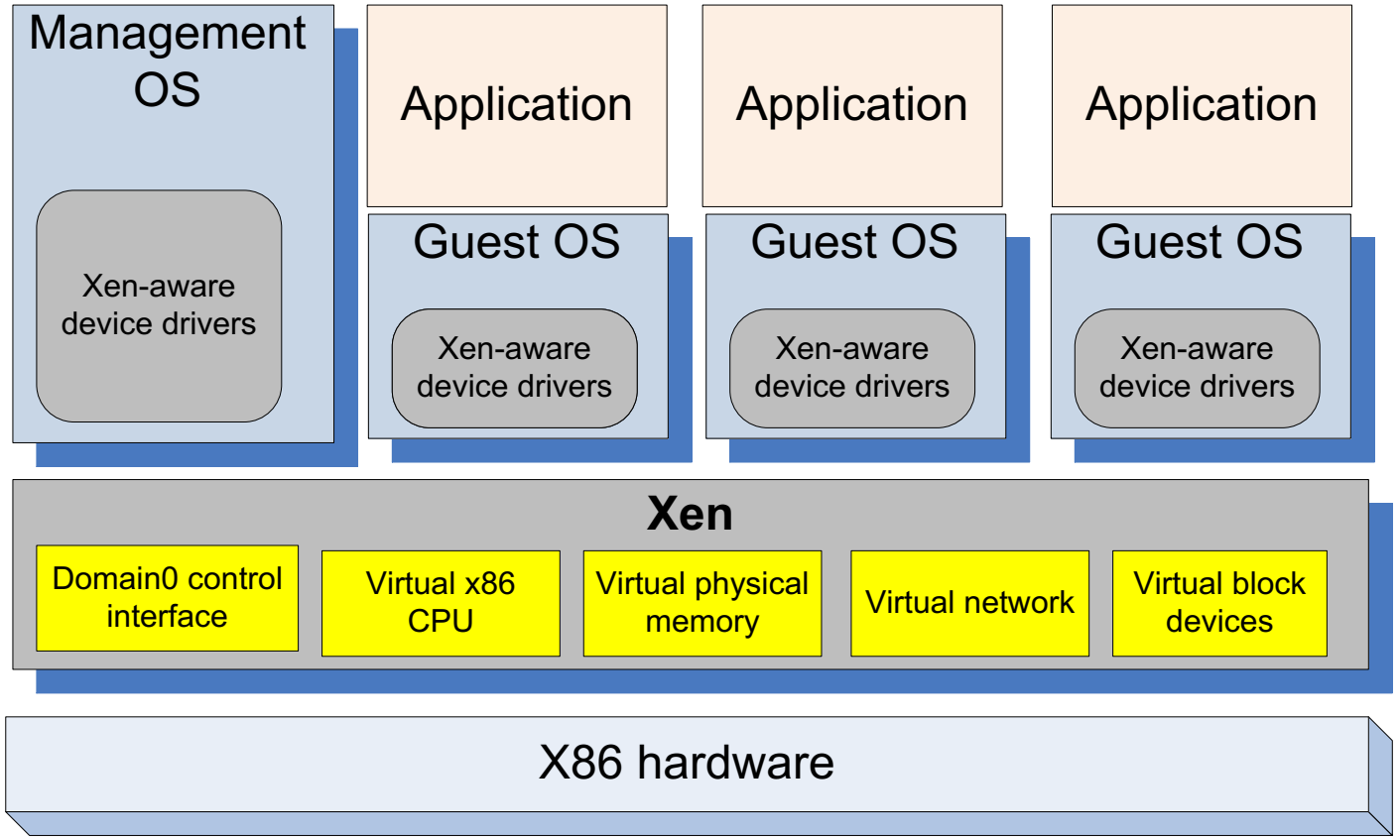

Concurrency
Scalability
(performance)
Openness
Resilience
(fail-over)
Security
Heterogeneity
Transparency
Ideal
Distributed
System

Cloud services - Responsibilities

"You" can use cloud services (infrastructure, platform or software) for your development



Is Cloud Computing a Distributed System?



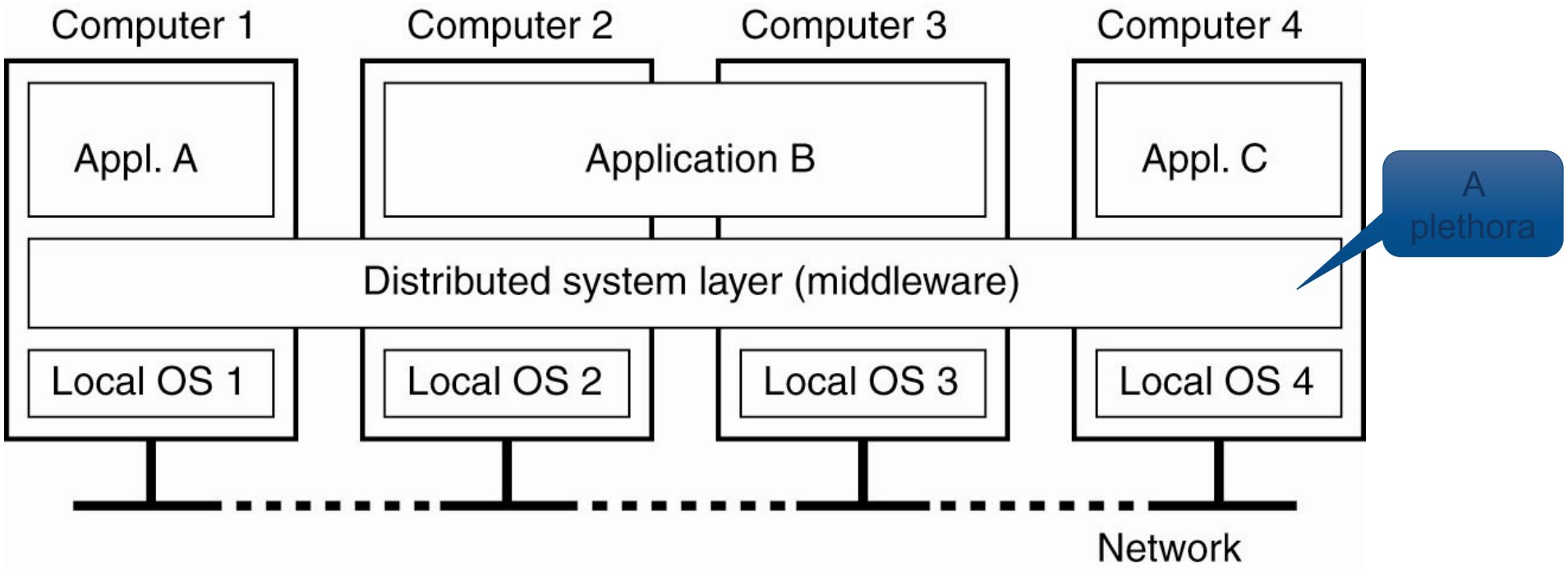
Yes!

- A cloud is typically a distributed system inside
  - Although geographically limited to a few datacenters
- The cloud has a security boundary
  - Strong firewalls
  - Only some front-end services/servers are exposed through the firewall
  - Once inside the cloud, those servers can communicate inside the security realm
- Fault tolerance and high availability are key properties of distributed systems (if designed right)

Distributed Systems topics

- Client-server and P2P paradigms
  - Concurrent execution and multiple nodes intercommunicate
- Remote procedure calls and message-based process communication
  - Key paradigms since 1980s in embedded systems
  - Communication is a key functionality in distributed systems.
- Coordination of distributed processes through noisy and unreliable communication channels
  - Messages lost, duplicated, distorted, or out of order
- High availability requirements drives distribution
  - Checkpoint-restart procedures (local recovery) facilitates to restart a process after system failure, but it is not seamless
  - For high availability (seamless failover) a system should be fully operational while some node is in the process of recovering.

A Distributed System



Examples of fundamental distributed services

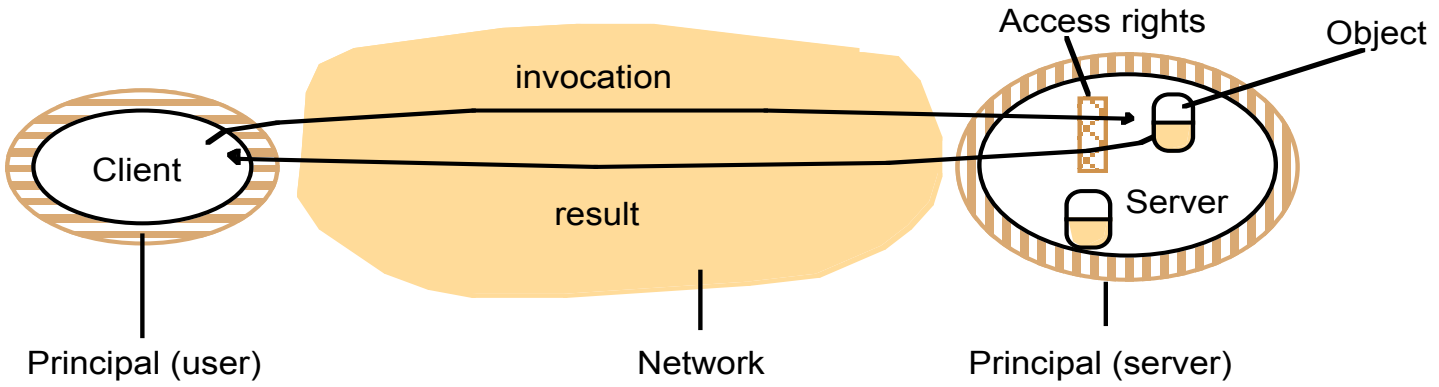
- Distributed memory
- Distributed objects ← focus in today's lecture
- Distributed messaging (e.g., pubsub, msg queues)
- Distributed structured storage (e.g.,file systems)
- Distributed (key,value) stores

Motivation for distributed systems

- Increased performance/cost ratio
  - Resource sharing
- Enhanced reliability
  - Availability, Integrity and Fault-tolerance
  - Access to remote data and resources, always there
- Scalability
  - Nodes can be added or removed
- Modularity and incremental expandability
  - Non-monolithic software (if designed right)

Distributed System Design - Direct communication

- Architectural model
  - Direct communication by: IPC, Request-Reply, RPC, *RMI*
  - *Distributed Objects* and the *client-server* paradigm



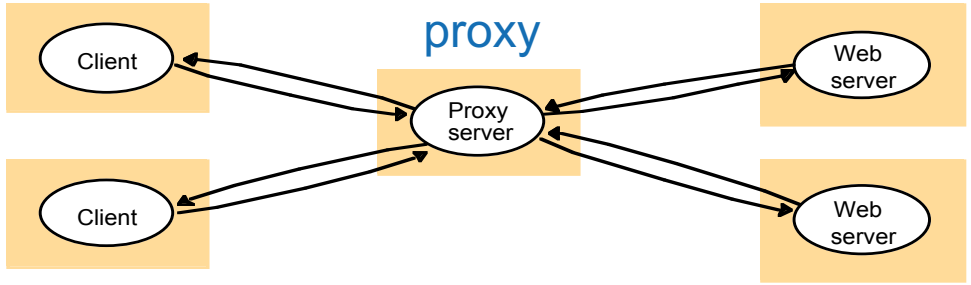
Client often called: service requester, or service consumer
Server often called: service producer

Clients can typically both read and write to objects

Distributed System Design - Indirect communication

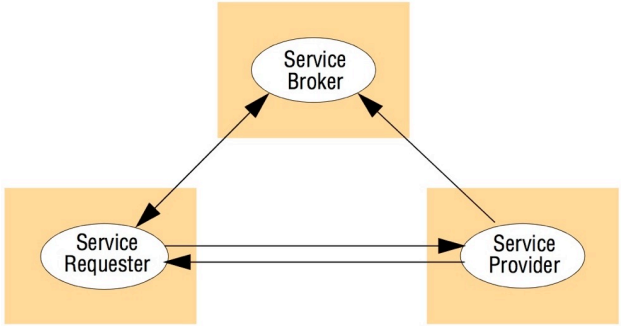
- Architectural model
  - Indirect communication by: Group communication, publish-subscribe, persistent tuple space, distributed shared memory
  - Layered, *tiered* (organize functionality of a layer and place it on to a node)

proxy



payload data goes through the intermediary

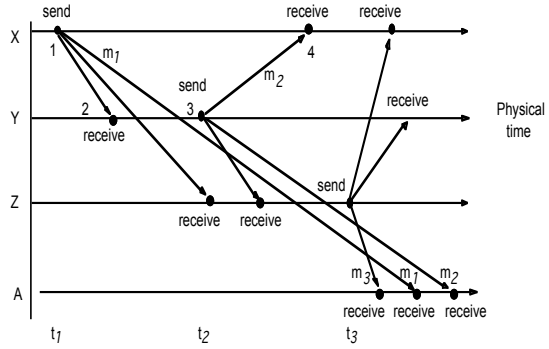
broker



Services are discovered in registry, then data goes directly

Distributed System Design

- The Interaction model includes performance of a communication channel, computer clocks and timing events.
  - Two variants, synchronous distributed systems
    - Time for executions has known lower and upper bounds
    - Each message has a known bound time
    - Each process has a clock drift with a known bound.
    - Common inside embedded systems
  - and asynchronous distributed systems
    - Process execution speed varies and can take arbitrary time
    - Unbound message translation time
    - Clock drift rates are arbitrary
    - Common in Internet/web-based systems



Content

- Course description
- Introduction to Distributed Systems
- Primitives for Distributed Communication
- Load balancing, web system use case
- Multiprocessor and Multicomputer Systems
- Event ordering and Consistency of a Distributed System
- Transparencies

Primitives for Direct Distributed Communication

- Synchronous
- Asynchronous
- Blocking
- Non-blocking

Primitives for Distributed Communication

- Send primitive (invoked by sender)
  - At least two parameters: destination, and record (e.g., buffer space) including the information.
- Receive primitive (invoked by receiver)
  - At least two parameters: the source from where data is received, and a record (e.g., buffer space) where to place received information.

Synchronous primitives

- Send and receive handshake with each other
- Send completes after that it learned that the corresponding receive has completed, that is when copied into receiver's user buffer
- Typically blocking senders and waiting receivers until handshake (aka rendez-vous) has completed
  - Block until data is copied from the user space to kernel space, sent over the network , received in receiver's user buffer and ACK received
- A send-receive event can be modeled as one atomic event on a timeline
  - A great advantage to ensure system correctness

Asynchronous primitives

- Typically, non-blocking for senders
  - The control returns after data has been copied out of user's buffer
  - In some cases, a handle is returned for checking if the call is completed
  - Periodical checking (loop)
  - Wait-call (handles multiple non-blocking primitives, blocks on call)
- On receiver side it is like synchronous
  - Blocks until message arrives
- Can provide much better performance than synchronous when round-trip times are non-negligible

Content

- Course description
- Introduction to Distributed Systems
- Primitives for Distributed Communication
- Load balancing, web system use case
- Multiprocessor and Multicomputer Systems
- Event ordering and Consistency of a Distributed System
- Transparencies

Dynamic Load Balancing on Web-server Systems

Source: Valeria Cardellini, Michele Colajanni, and Philip S. Yu

Modified by Olov Schelén



Distributed web-server system

- Service model: client-server or client-cloud
  - Clients may be behind NAT/firewall, but bidirectional holes for http are automatically punched when they reach out to services
  - This is one technical reason why the cloud model is popular (peer-to-peer direct communication is more complicated w r t NATs)
  - There are obviously also business reasons
    - This is really the major thing…
    - it is much easier to “know” what customers are doing
    - and upstream bw is costly in mobile
    - … etc

Distributed Web-server System

- Consisting of multiple Web-server hosts, distributed on LANs and WANs
  - These are front end servers that are essentially stateless or at most have session state
  - Persistent data resides in replicated back-end database servers
- Spread incoming requests among these servers
- Each server can respond to any request
- Successful load-balancing approaches make the distributed system appear as a single service or a single host

Distributed Web-server Architectures

- Web-server based dispatching & fail-over
  - DNS-based
  - Dispatcher-based
  - Server-based
- Client based dispatching & fail-over
  - Web clients
  - Client-side proxies

DNS-based approach

- Architecture transparency, clients see which server they talk to.
- DNS can advertise any of the available servers

```
olov@B5995 ~ % nslookup aftonbladet.se
Server:         130.240.19.2
Address:        130.240.19.2#53

Non-authoritative answer:
Name:   aftonbladet.se
Address: 35.71.185.87
Name:   aftonbladet.se
Address: 52.223.48.227
```

- DNS has limited control on requests reaching the Web cluster
  - There is a Time To Live (TTL) for DNS mapping, telling how long they should be valid in DNS caches

DNS load balancing

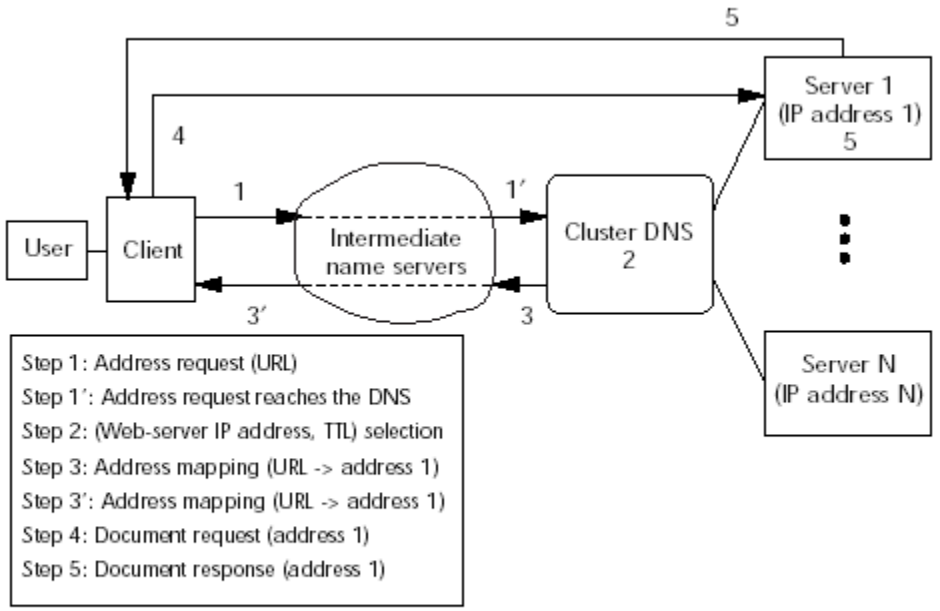


Figure 1. DNS-based approach to load balancing.

Basic DNS

- The DNS server response contains a list of IP-addresses
  - With Round Robin DNS, the list is “rotated” for each response
  - The client (web browser) should pick the first one
  - However, when using “netmask ordering” the DNS server prioritizes the list so that numerically close servers are early in the list
  - Very coarse and load-insensitive load balancing
    https://en.wikipedia.org/wiki/Round-robin_DNS
- The client can fail-over to the next address in the list when a web server does not respond.
  - Provides slow (30 seconds?) geographical fail-over. However, server-side session state (if any) is typically lost. And the server may not accept to receive current client-side state (forms etc.)
- However, this is controlled by the client (i.e., servers may not control it)
  - Lately some browsers (e.g., chrome) arbitrate (load-balance) between all advertised addresses (i.e., no universal fail-over order)
- A typical cloud makes an internal separation
  - front-end servers, typically stateless, or keeping only session state
  - Back-end servers (databases etc.), that have persistent state

Load-balancing (scheduling) algorithms

- Constant DNS TTL algorithms
  - System-stateless algorithms
    - Basic Round-Robin DNS
  - Server-state-based algorithms
    - The DNS server monitors the web-server load and reorders the list
    - Very slow control if DNS TTL is high
    - Loads the DNS if TTL is short
  - Client-state-based algorithms
    - Hidden load weight
      - Multitier round-robin policy
    - Consider relative server-to-client topology and client-to-server link latency
    - Network proximity information, round trip delays

Typical DNS cluster of today, supporting failover

- DNS provider has multiple root servers, geographically distributed (DNS hosting with high availability roots)
- Several server addresses can be registered for a given service
  - But only one (or a few) at the time is advertised, with relatively short TTL
- The DNS provider tests the advertised server address(es) regularly (e.g., ping, heart-beat)
  - If unavailable, it switches to advertise another registered one
- Advantages: reasonable quick failover, controlled by the DNS server(s) (clients only see operational ones)
  - Examples: DynDNS, DNSmadeeasy

Only one IP address returned by DNS

- There are other similar approaches using special purpose dispatchers (load balancers), or combinations
- In any case it looks like this:

```
olov@B5995 ~ % nslookup google.com
Server:         130.240.19.2
Address:        130.240.19.2#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.74.142
```

Question?

- You have a hobby service that should be always available, but your internet connection is not reliable enough.
- How would you go about to setup a small “hobby” web system with two geographically separate web servers?

Dispatcher-based Approach

- Centralized request scheduling
  - Packet address rewriting
  - Packet forwarding
  - HTTP redirection
- Requests, and typically all following per-session traffic are routed through this device
  - At least in one direction (single-rewriting)
  - But typically, in both directions (double rewriting)
  - The service then appears to be coming from one host

Packet double-rewriting

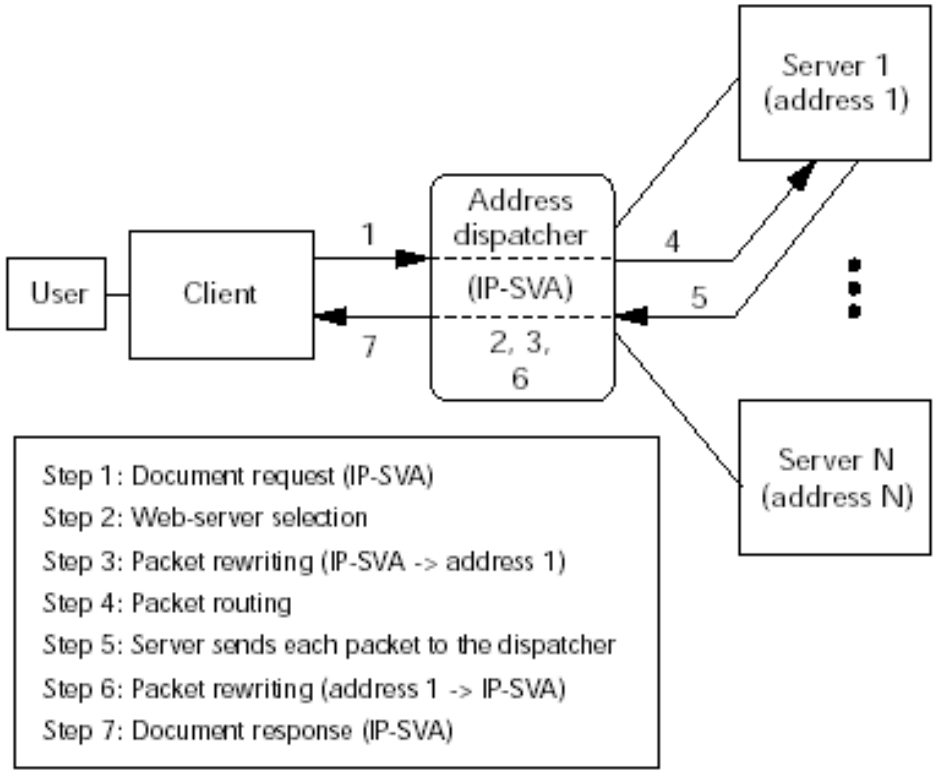


Figure 3. Packet double-rewriting by the dispatcher.

Compare setting up Port forwarding in your Home gateway: similar but typically no load balancing there

Observation

- Dispatcher-based solutions are efficient and uncomplicated for clients and DNS servers
  - In the basic form only one IP address is needed
- However, typically you want a redundant internet connection where the network operator provides dynamic seamless fail-over
  - E.g., multiple fibers or other connections without faith sharing
  - Datacenters typically have this
  - If you like to have geographical failover, using different network operators, combinations are needed

Multi-level packet forwarding

- Network Dispatcher
  - Works with LANs and WANs
  - Client and server transparent
- ONE-IP Address
  - Routing-based dispatching
  - Broadcast-based dispatching

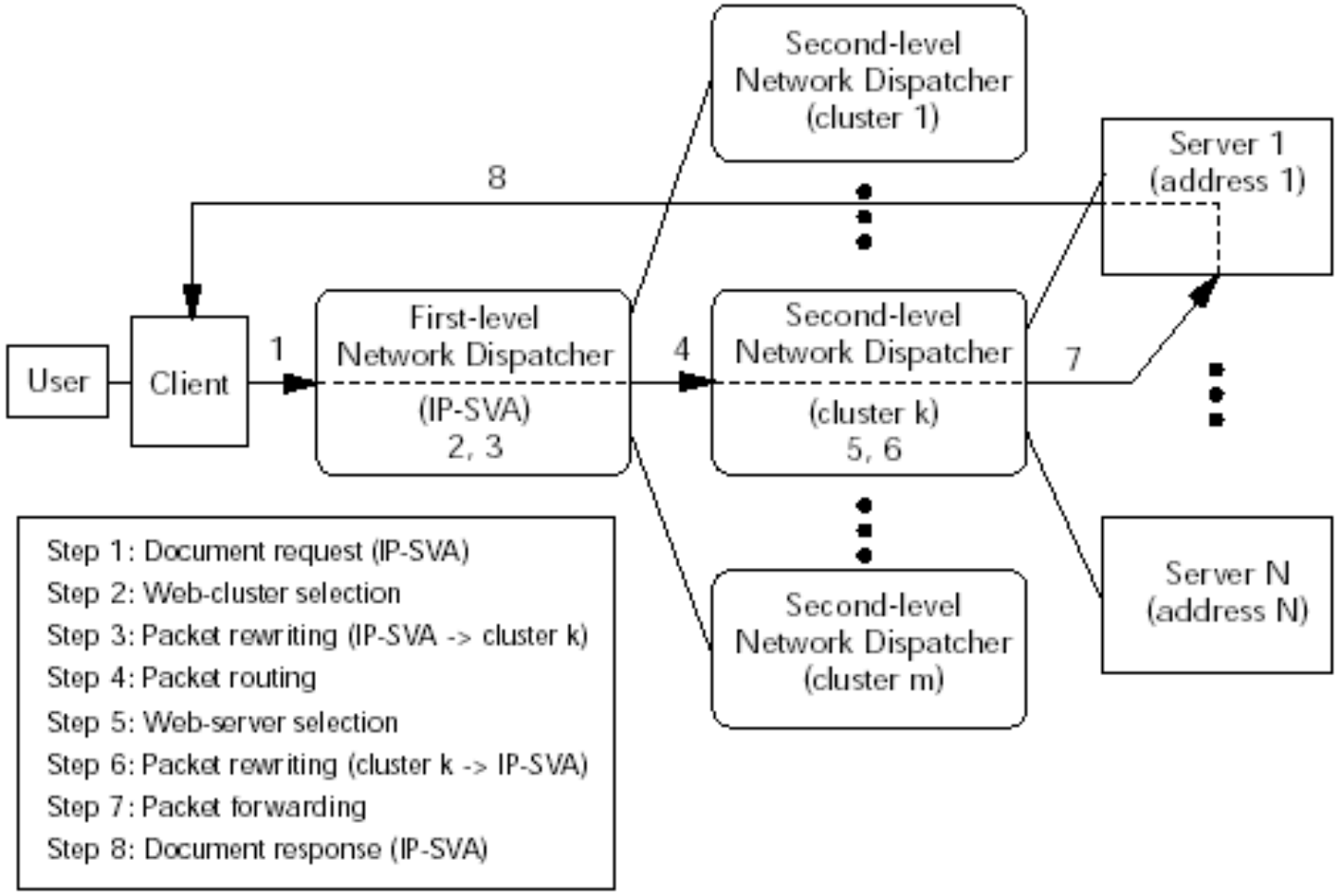


Figure 4. WAN Network Dispatcher architecture.

Content

- Course description
- Introduction to Distributed Systems
- Primitives for Distributed Communication
- Load balancing, web system use case
- Multiprocessor and Multicomputer Systems
- Event ordering and Consistency of a Distributed System
- Transparencies

Relation to multiprocessor systems

- A multiprocessor system
  - Multiple CPUs
  - Direct access to shared memory
  - Usually corresponds Uniformed Memory Access (UMA)
    - Access latency the same for all CPUs
    - Inter-process communication through shared memory, or through message-based primitives (which are implemented by shared memory)

- Note: each node in a geo distributed system typically executes on a multiprocessor host

Processing modes
Michael Flynn's classification

- Single instruction stream, single data stream (SISD), one processor/core, single RAM
- Single instruction stream, multiple data streams (SIMD), vector processing, several homogeneous processors, lock-step parallel execution over arrays/matrixes
- Multiple instruction streams, multiple data streams (MIMD), multiple processors that operate asynchronously.
  - This is them mode in standard operating systems (also adding multiple threads inside each process to allow parallel execution with fewer context switches)
  - This is the mode of operations in distributed systems

Concurrency; race conditions and deadlocks

- Concurrent execution can be challenging.
  - Race conditions, an undesirable effect when the results of concurrent execution depend on the sequence of events.
  - Shared resources must be protected by locks/ semaphores /monitors or serialized message systems to ensure serial access.
  - Deadlocks and live-locks are possible.
- The four Coffman conditions for a deadlock:
  - Mutual exclusion - at least one resource must be non-sharable, only one process/thread may use the resource at any given time.
  - Hold and wait - at least one processes/thread must hold one or more resources and wait for others.
  - No-preemption - the scheduler or a monitor should not be able to force a process/thread holding a resource to relinquish it.
  - Circular wait - given the set of n processes/threads $\{P_1, P_2, P_3, \ldots, P_n\}$. Process P_1 waits for a resource held by P_2, P_2 waits for a resource held by P_3, and so on, P_n waits for a resource held by P_1.

Race conditions is your concern!

- In your lab assignment
- There are always some trade-offs
  - Course granular locking (e.g., serializing all operations on a critical datastructure through only one thread).
    - Little parallelism, simple, works, but slow
  - Fine granular locking (e.g., lots of threads work concurrently on different parts of a datastructure)
    - Lots of parallelism, can be blasting fast
    - Prone to race conditions if you make any small mistake in you code
- Advice
  - keep initial solutions simple and functionally correct (not prone to race conditions, a k a thread safe)
  - Understand and present your trade-offs! Have a plan for performance improvements

Granularity of a program

- The ratio of computation and communication
- Coarse grained granularity => productive CPU cycles >> CPU cycles for communication
- Fine grained granularity => productive CPU cycles << CPU cycles for communication
- You must also consider the speed/cost of communication when building an efficient system
  - Local communication (e.g., between processes or threads on the same computer) is MUCH faster than communication across the internet.
  - Maximize parallelism for slow/costly operations (e.g., communication to other nodes over the IP network)

Granularity of a program

- Coarse grained granularity
  - Large blocks of code can be executed in parallel without the need to communicate or synchronize with others processes or threads
  - Best suited for loosely coupled systems
    - Distributed systems and MIMD computers
- Fine grained granularity
  - Relatively small block of code executed without the need to communicate or synchronize with others processes or threads
  - Best suited for tightly coupled systems
    - Embedded systems, and SIMD and MISD computers
    - Communication round-trips hits performance

Operating Systems

- Multiprocessor operating systems
  - Tightly coupled processors or CPU cores
  - Tightly coupled software (runs identical copies of the operating system)
  - Typically supports both processes and threads

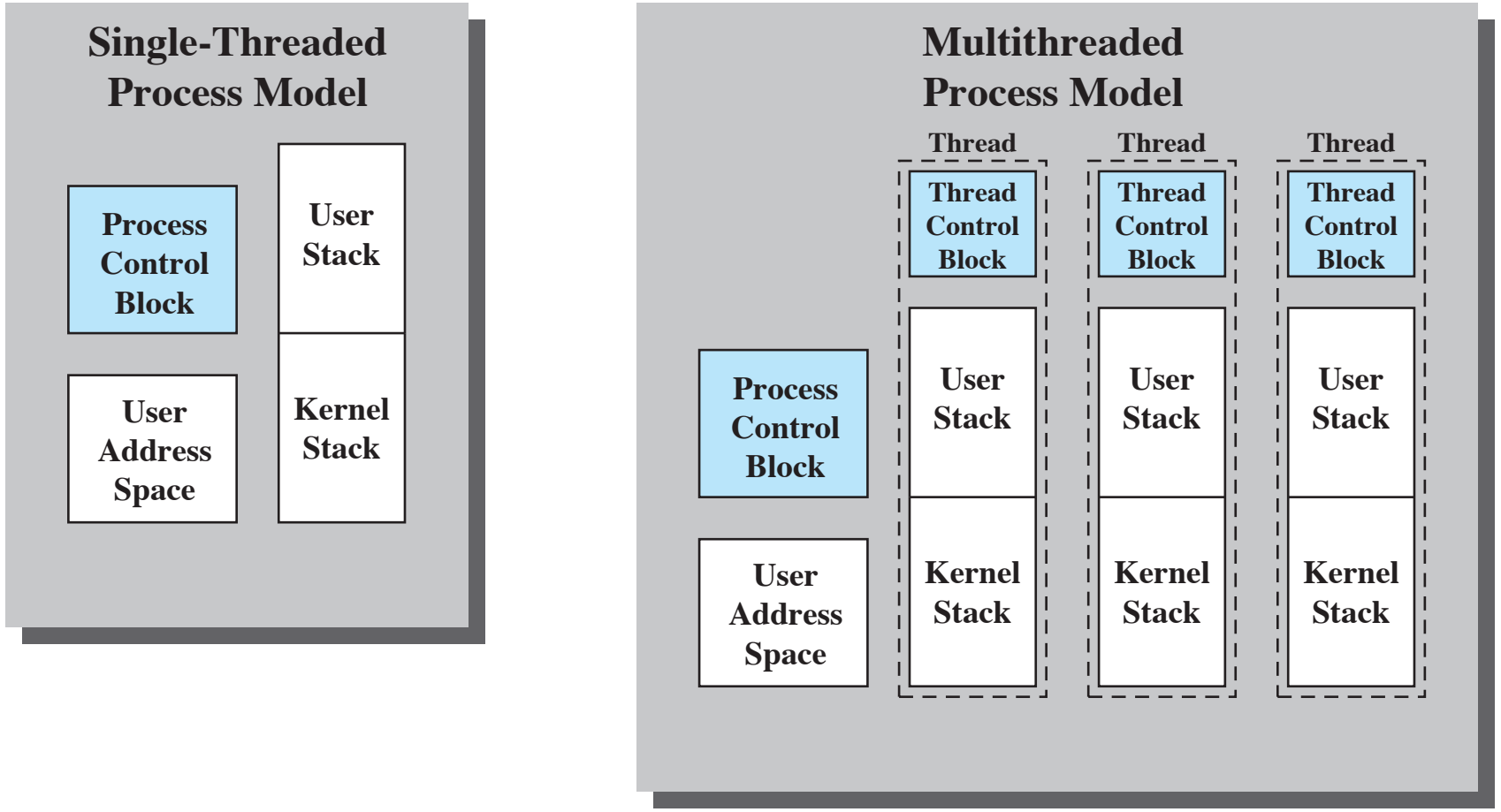


Figure 4.2 Single Threaded and Multithreaded Process Models

Processes and threads

- Each process has its own protected address space
  - A process can not fiddle with RAM in other processes
- There can be multiple threads executing in parallel (mapped to different cores)
  - These threads share the address space of their process
- Process context switching means changing address space, caches must be flushed, etc
- Thread switching inside a process does not require any change of address space – much faster

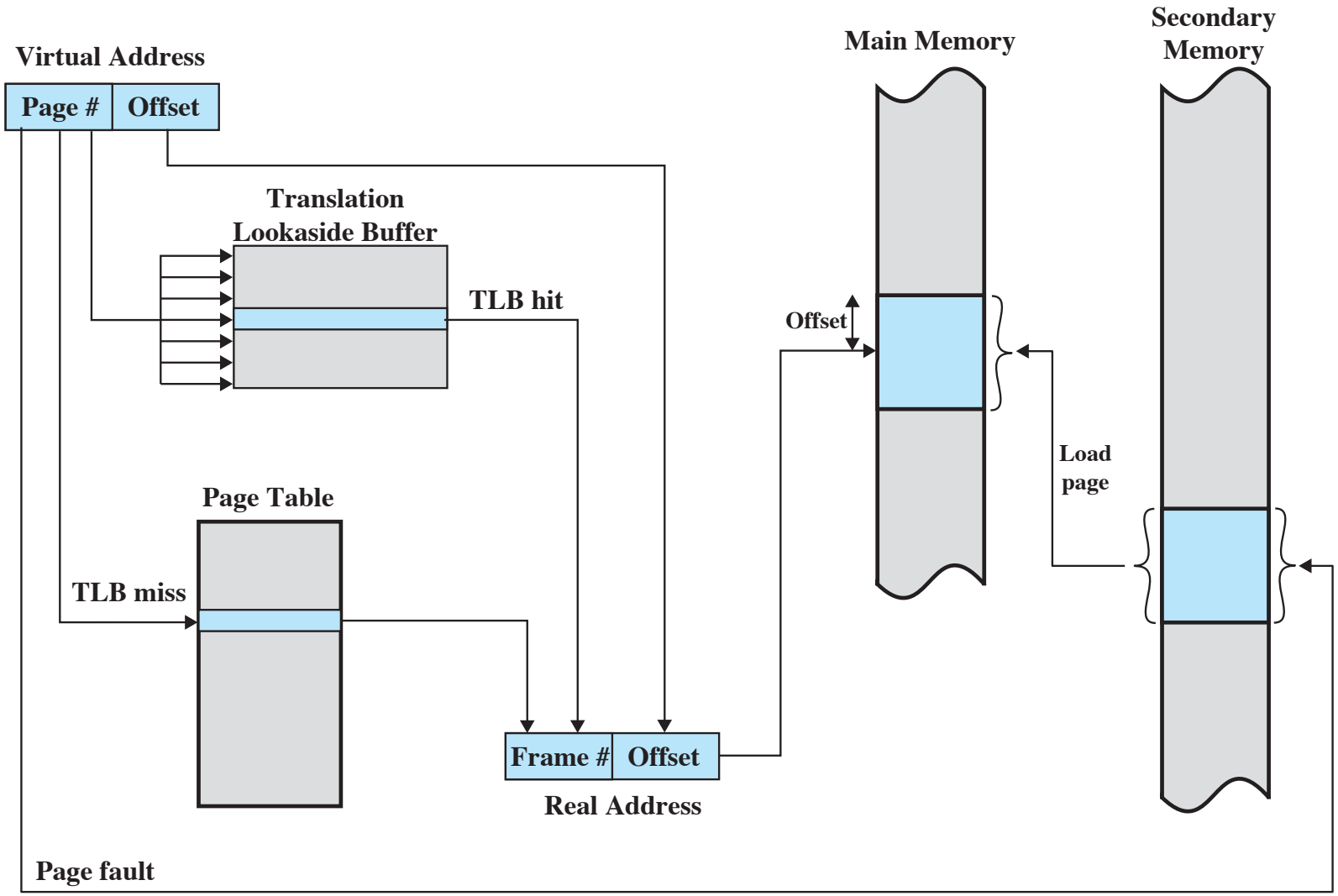


Figure 8.6 Use of a Translation Lookaside Buffer

The TLB is a cache

- At context switch (process switch), the TLB must be “Flushed”.
- The performance hit of a context switch
  - TLB is empty
  - Page table (OS managed) may be on disk
  - Data (Process managed) may be on disk
  - Code (shared read only) may be on disk
- However, *threads* inside a process do share an address space, i.e., no TLB flushing at thread switch.
- However, even at process switching some entries could be saved
  - Referring to shared read only data (e.g., code)
  - Referring to other data that is supposed to be shared memory

Thread challenges

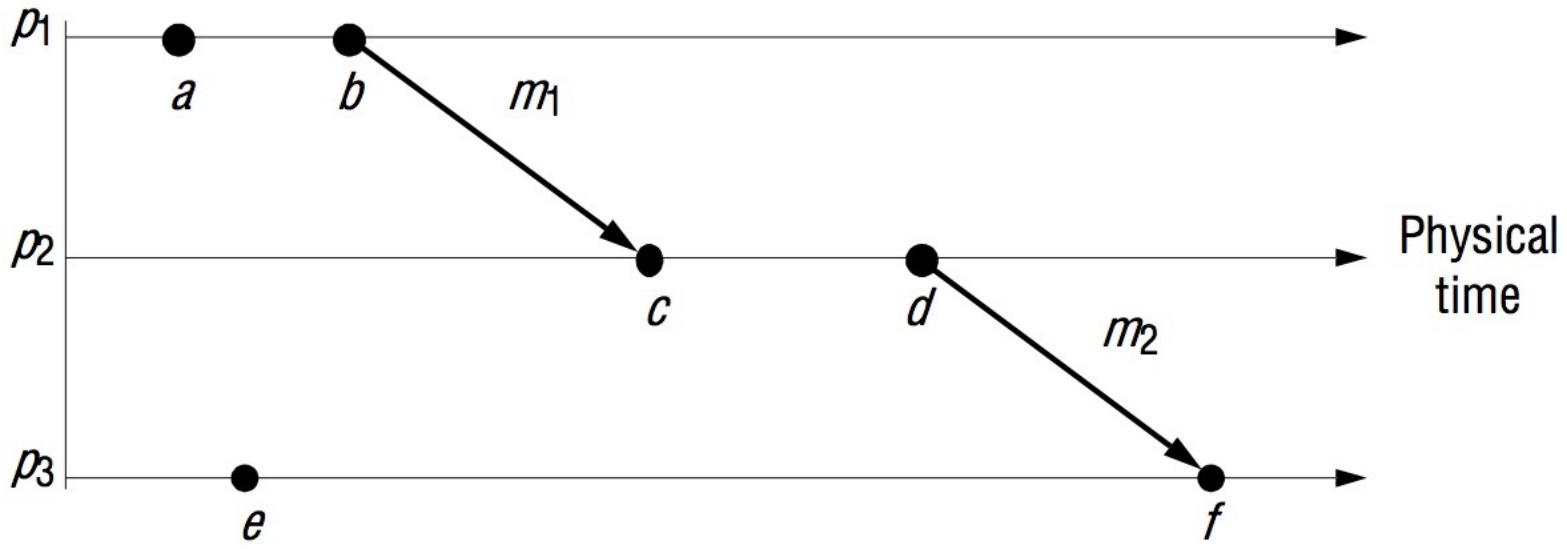
- The application (or system) programmer must design the software for multi-threading
  - This is truly a big challenge
- Ref: http://randu.org/tutorials/threads/
- Not all problems are apt for multi-threading
  - Preferable for problems where fairly independent parallel processing can be done
  - Typically, not preferable for contention type problems that require a lot of mutual exclusion and blocking, potentially causing race conditions.
- There is no such thing as a free lunch…

Content

- Course description
- Introduction to Distributed Systems
- Load balancing, web system use case
- Multiprocessor and Multicomputer Systems
- Primitives for Distributed Communication
- Event ordering and Consistency of a Distributed System
- Transparencies

Ordering

- How to decide that *a* happened before *c* ?
- Process coordination requires
  - A global concept of time by entities
  - A measurement of time intervals

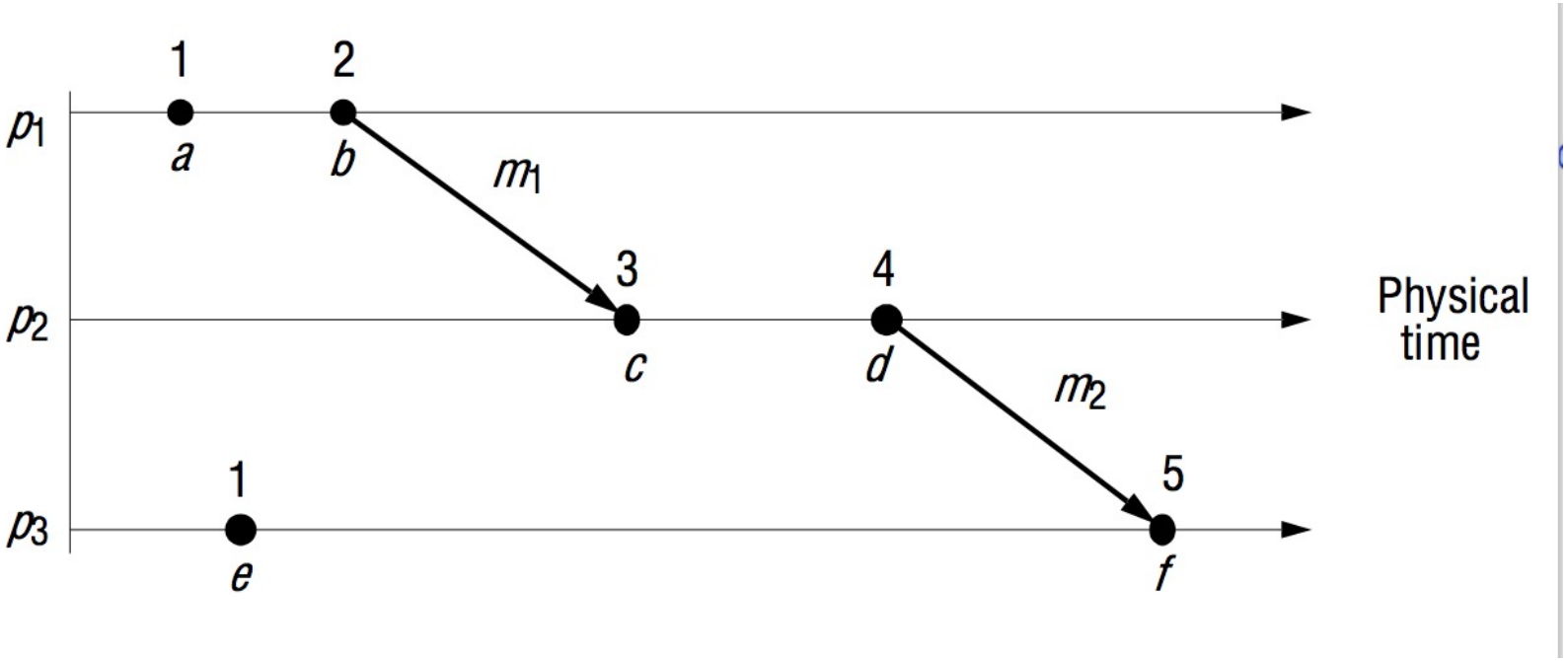


Ordering using Logical clocks

- Logical clock (LC) → an abstraction necessary to ensure the clock condition in the absence of a global clock.
- A process maps events to positive integers. LC(*e*) the local variable associated with event *e*.
- Each process time-stamps each message *m* it sends with the value of the logical clock at the time of sending:
  - TS(*m*) = LC(send(*m*)).
- The rules to update the logical clock:
  - LC(*e*) = LC + 1 if *e* is a local event
  - LC(*e*) = max (LC, (TS(*m*)+1)) if *e* is a *receive* event.

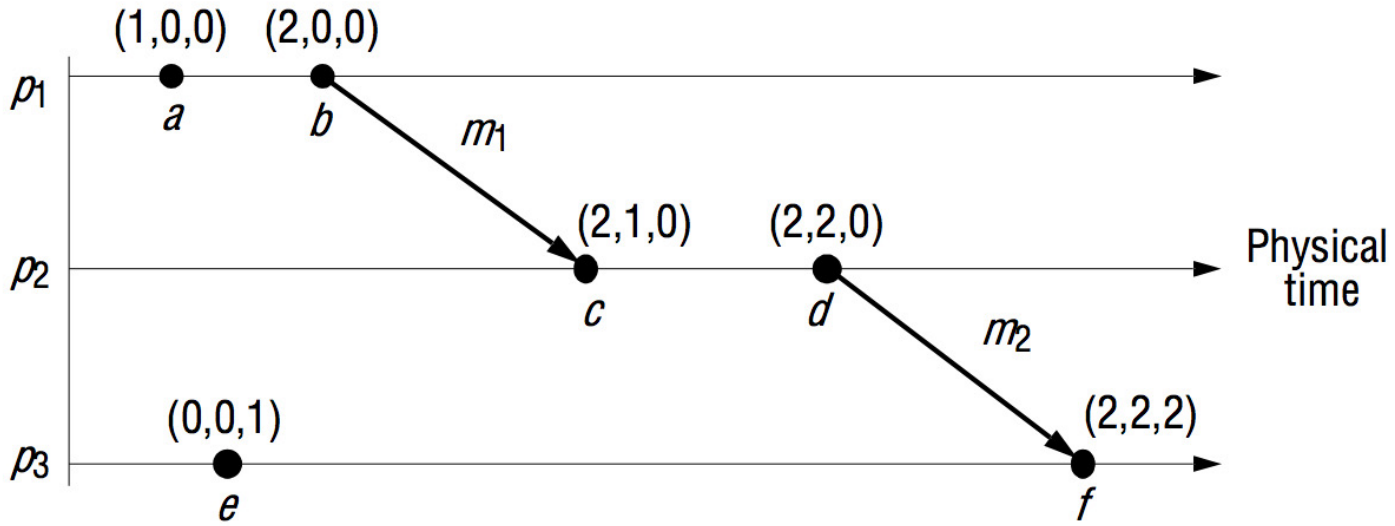
Casual ordering

- Lamport's logical clock
  - Use a counter that increases by one each event
  - Included in messages sent



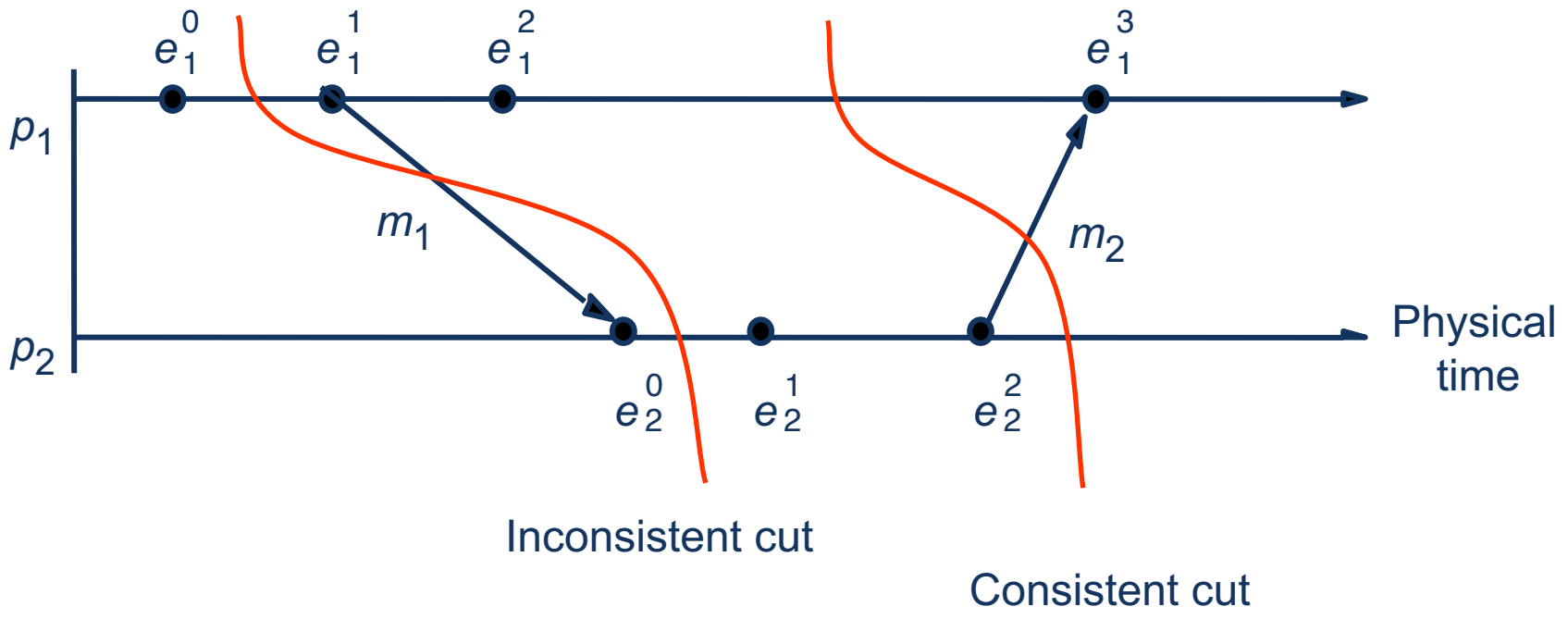
Partially ordered

- Vector clocks
  - Avoids the drawback with Lamport that we cannot conclude that e -> é if LC(e) < LC (é) (why?)
  - A vector consisting of a time stamp for each process
  - Time vector sent with each message



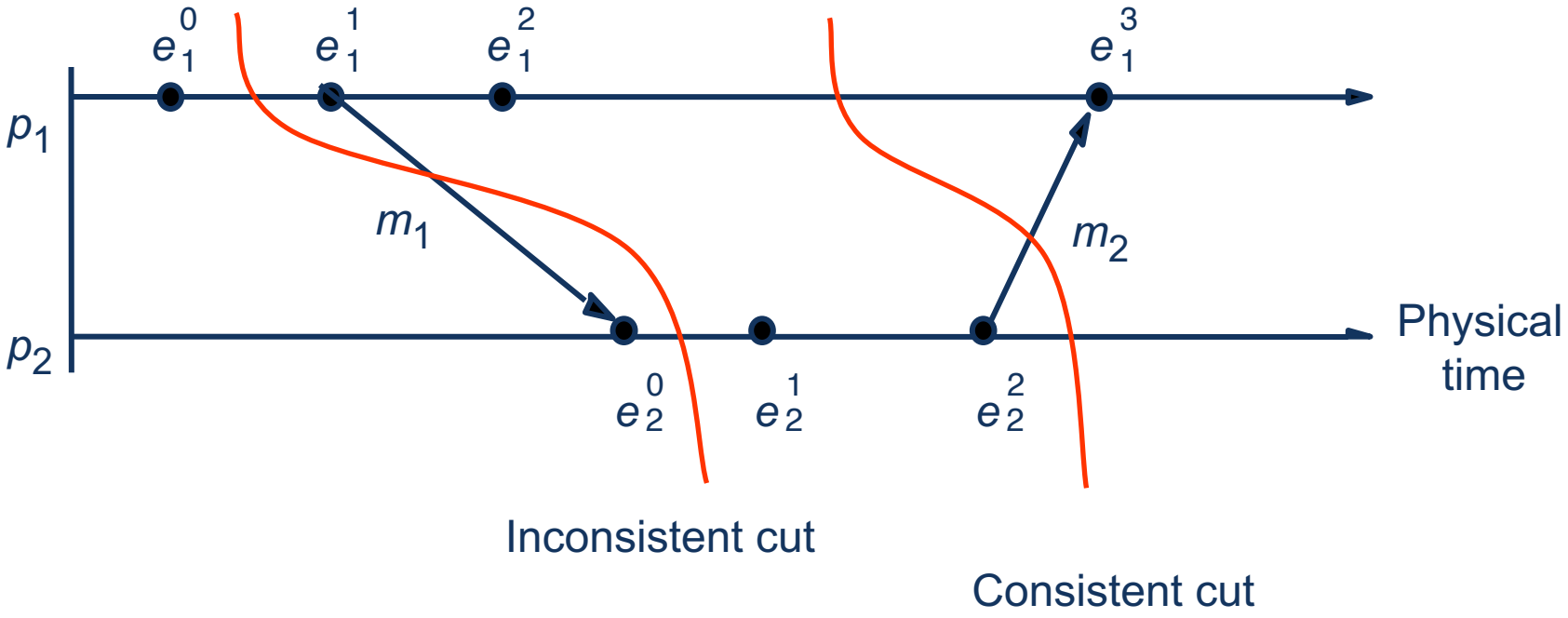
Any limitation?

Global state and consistent cut of a Distributed System



Global state of a Distributed System

- Both processes and channels have states
- If p_1 sent a message to p_2, but not received, the channel has a state that m belongs to its state



Content

- Course description
- Introduction to Distributed Systems
- Load balancing, web system use case
- Multiprocessor and Multicomputer Systems
- Primitives for Distributed Communication
- Event ordering and Consistency of a Distributed System
- Transparencies

Transparencies

- Access transparency
  - enables local and remote resources to be accessed using identical operations.
- Location transparency
  - enables resources to be accessed without knowledge of their physical or network location (for example, which building or IP address).
- Concurrency transparency
  - enables several processes to operate concurrently using shared resources without interference between them.

Transparencies

- Replication transparency
  - enables multiple instances of resources to be used to increase reliability and performance without knowledge of the replicas by users or application programmers.
- Failure transparency
  - enables the concealment of faults, allowing users and application programs to complete their tasks despite the failure of hardware or software components.

Transparencies

- Mobility transparency
  - allows the movement of resources and clients within a system without affecting the operation of users or programs.
- Performance transparency
  - allows the system to be reconfigured to improve performance as loads vary.
- Scaling transparency
  - allows the system and applications to expand in scale without change to the system structure or the application algorithms.

Discussion and Questions

